Session 1: Reading, Showing, Resizing, Cropping, Rotating

Notebook: `Session1/Session1.ipynb`

Written for someone starting computer vision for the first time. Part 1 is the background you need. Part 2 walks through the notebook, with the answers to the questions that came up while studying it folded into the matching sections. Part 3 is a quick Python OOP refresher.

Order used here

1. Background: pixels, coordinates, NumPy, slicing, interpolation, rotation math
2. The notebook: setup and paths → reading → showing → resizing → cropping → rotating
3. Python OOP refresher
4. Cheat-sheet

Part 1: Background you need first

1. What an image is to a computer

A digital image is a **grid of tiny squares called pixels**. Each pixel holds numbers that describe its color.

- **Grayscale image:** each pixel is **one number** from 0 (black) to 255 (white). The whole image is a 2D table: `height × width`.
- **Color image:** each pixel is **three numbers**, one each for Blue, Green and Red. The image is a 3D block: `height × width × 3`. Each of those 3 layers is called a **channel**.

💡 **Why 0 to 255?** Each value is stored in 1 byte (8 bits), and 8 bits can hold 2^8 = 256 values. That data type is called `uint8` (unsigned 8-bit integer). You'll see `dtype=uint8` in the notebook.

⚠️ **OpenCV stores colors as BGR, not RGB.** If you show an OpenCV image with another library like matplotlib, blue and red swap until you convert it with `cv.cvtColor(img, cv.COLOR_BGR2RGB)`.

2. Coordinates in an image

The origin (0, 0) is the **top-left corner**, not the bottom-left like in math class.

- **x** is the column number and grows to the **right**.
- **y** is the row number and grows **downward**.

```
(0,0) ──────────► x (width / columns)
  │
  │
  ▼
  y (height / rows)
```

3. NumPy (essential)

NumPy is a Python library for working with big grids of numbers quickly. Its main object is the `ndarray` (n-dimensional array).

In OpenCV-Python, an image is just a NumPy array. That's why the notebook prints `<class 'numpy.ndarray'>`.

```
import numpy as np
a = np.array([[1, 2, 3],
              [4, 5, 6]])
a.shape      # (2, 3) → 2 rows, 3 columns
a.dtype      # int64 (type of the numbers)
a[0, 1]      # 2 → row 0, column 1
a[0:2, 1:3]  # rows 0 to 1, columns 1 to 2 → [[2,3],[5,6]]
```

For an image, `img[y, x]` gives the pixel at row `y` and column `x`. In a color image that pixel is `[B, G, R]`.

4. Python slicing (used for cropping)

`start:stop` means "from `start` up to but **not including** `stop`."

```
lst = [10, 20, 30, 40, 50]
lst[1:4]   # [20, 30, 40]
lst[:2]    # [10, 20] (start omitted = from the beginning)
lst[3:]    # [40, 50] (stop omitted = to the end)
```

NumPy allows slicing in several dimensions at once, separated by commas: `img[rows, cols]`.

5. Tuple unpacking

```
h, w, _ = img.shape   # shape is (300, 332, 3)
# h = 300, w = 332, and _ = 3
```

The `_` is a Python convention for "I don't need this value."

A **tuple** is a group of values in plain parentheses, like `(200, 300)`. It's how you pass a size to OpenCV.

6. Other small things used

- `import cv2 as cv`: imports OpenCV and gives it the short name `cv`.
- `assert condition, "message"`: if the condition is False, the program stops with that message.
- `int()`: turns **one** value into a whole number by cutting off the fractional part. Pixel counts must be whole numbers.
- **Keyword arguments:** `f(center=center, angle=45)` names each argument explicitly, so their order doesn't matter.
- **String concatenation:** `+` between two strings joins them: `'abc' + 'def'` → `'abcdef'`.

7. Interpolation (needed for resizing)

When you resize an image, the new pixels don't line up exactly with the old ones, so OpenCV has to **estimate** their values. That estimation is called **interpolation**. How each method works is explained in Part 2, Section D.

Flag	Idea	Best for
`INTER_NEAREST`	Copy the closest pixel	Fastest, but looks blocky
`INTER_LINEAR`	Weighted average of the 4 nearest pixels	Good default
`INTER_AREA`	Averages the area each new pixel covers	**Shrinking** images
`INTER_CUBIC`	Uses 16 neighboring pixels, smoother	**Enlarging** images (slower)

8. Aspect ratio

Aspect ratio is `width / height`. If you **preserve** it, the image keeps its proportions. If you don't, the image gets stretched or squished.

9. Rotation math, lightly

To rotate a point by an angle θ around the origin, you multiply it by this matrix:

```
[ cosθ  -sinθ ]
[ sinθ   cosθ ]
```

You don't need to memorize it. The key idea: **a transformation (rotate, move, scale) can be written as a matrix, and applying it to every pixel's coordinates transforms the whole image.** OpenCV builds the matrix for you (`getRotationMatrix2D`) and applies it for you (`warpAffine`).

💡 "Affine" means straight lines stay straight and parallel lines stay parallel. Rotating, scaling, moving and shearing are all affine transformations.

Part 2: The notebook, section by section

A. Setup and paths

```
import cv2 as cv
path = './images/'
cv.__version__        # '4.10.0'
```

This imports OpenCV and stores the **folder** where the images live.

How does `path` lead to the image?

`path` never holds the image's full path. It only holds the folder. The full path is built when the image is read:

```
img = cv.imread(path + 'dog.jpg')
# './images/' + 'dog.jpg'  →  './images/dog.jpg'
```

`+` joins the two strings, so OpenCV receives `'./images/dog.jpg'`. The `path` variable itself stays `'./images/'`. Storing just the folder lets you reuse it: `path + 'cat.png'`, `path + 'car.jpg'`, and so on.

What `./` means

- `.` is the **current working directory**: a shortcut so you don't have to type its full path.
- `./images` is **a folder named `images` inside the current directory**. The name `images` isn't special; Python just looks for a folder with that exact name.
- `./images` and `images` are the same. The `./` only makes "start from here" explicit.
- `../` means the **parent** folder, one level up.

```
Session1/              ← current directory (.)
├── Session1.ipynb
└── images/            ← ./images
    └── dog.jpg        ← ./images/dog.jpg
```

Relative vs. absolute paths. These point to the same file:

```
'./images/dog.jpg'                                  # relative: from where Python runs
'C:/Users/.../skyexperts CV training/Session1/images/dog.jpg'   # absolute: full address
```

Relative paths are shorter and still work if the project folder moves or someone else runs it.

⚠️ "Current directory" means **where Python is running**, not necessarily where the file is. In Jupyter it's normally the notebook's folder. Check it with:

```
import os
os.getcwd()
```

⚠️ **The trailing `/` matters.** It separates the folder from the file name:

```
'./images'  + 'dog.jpg'   →  './imagesdog.jpg'     ❌ no such file
'./images/' + 'dog.jpg'   →  './images/dog.jpg'    ✅
```

`os.path.join('./images', 'dog.jpg')` adds the slash for you.

B. Reading an image

```
img = cv.imread(path + 'dog.jpg')
```

`cv.imread(path, flag)` loads the file into a NumPy array.

- flag `1` (the default) loads it in color (BGR).
- flag `0` loads it in grayscale.

⚠️ If the path is wrong, `imread` **doesn't raise an error**. It quietly returns `None`. That's why the next cell exists:

```
assert img is not None, "Cant read image"
```

```
print(type(img))  # numpy.ndarray → an image is just an array
img               # prints the raw pixel numbers
img.shape         # (300, 332, 3) → height 300, width 332, 3 channels
```

Reading the printed array

The first three numbers, `[89, 89, 89]`, are the **first pixel**: the top-left corner (row 0, column 0). The three numbers are its B, G, R values.

```
[            ← the whole image
 [           ← row 0 (the top row of pixels)
  [89,89,89] ← pixel at row 0, column 0
  [88,88,88] ← pixel at row 0, column 1
  [88,88,88] ← pixel at row 0, column 2
  ...
```

```
img[0, 0]   # array([89, 89, 89])
img[0, 1]   # array([88, 88, 88])
```

💡 Every pixel here has B = G = R. Equal channels mean gray, so this dog photo is actually black and white, even though it was loaded as a 3-channel image.

C. Showing an image

```
cv.imshow("Dog image ", img)   # opens a window with a title
k = cv.waitKey(0)              # waits for a key press (0 = wait forever)
cv.destroyAllWindows()         # closes the windows
```

- `waitKey` is **required**. Without it, the window appears and vanishes instantly, or freezes. It returns the code of the key you pressed, stored in `k`.
- In Jupyter, the window pops up **separately** and may appear behind your editor.

Why does the cell take over a minute when the image appears instantly?

The time goes to **`cv.waitKey(0)`**, which pauses until a key is pressed. The cell keeps running as long as the window is open, so the timer measures how long it took you to press a key.

Line	What it does	Time
`cv.imshow(...)`	Opens the window with the image	Instant
`cv.waitKey(0)`	Waits until you press a key	**Until you press a key**
`cv.destroyAllWindows()`	Closes the window	Instant

- To make it close on its own, give a time in milliseconds: `cv.waitKey(3000)` waits at most 3 seconds.
- To finish the cell, **click on the image window and press any key**. The key must go to the image window, not to VS Code.
- ⚠️ Don't close the window with the X button. The cell can keep waiting and the kernel may hang.

D. Resizing

```
cv.resize(src, dsize, fx=..., fy=..., interpolation=...)
```

You give it **either**:

- `dsize`: an exact output size, **(width, height)**, **or**
- `dsize=None` plus `fx` and `fy`: scale factors (for example 0.5 = half size).

⚠️ Bug in the notebook: width and height swapped

`cv.resize` expects `dsize` as **(width, height)**. The notebook does:

```
dim = (h, w)   # ❌ height first
```

The original is `(300, 332, 3)` (h=300, w=332), and the "60% downscaled" image comes out `(199, 180, 3)`: height 199, width 180. Height and width got **swapped**, so the image is distorted too. The correct

version:

```
dim = (w, h)   # ✅ OpenCV wants width first
```

💡 **NumPy's `.shape` is (height, width), but OpenCV's size arguments are (width, height).** It trips up everyone.

The same swap affects the "resize only width" and "resize only height" cells: in each one, the dimension that actually changes is the opposite of the title.

The individual resize cells

1. **Downscale (60%):** multiply both h and w by 0.6 → smaller image, same proportions (once the bug is fixed). Uses `INTER_AREA`, the right choice for shrinking.
2. **Upscale (200%):** same idea with 2×. (The heading says "< 100", but it means > 100.) It uses `INTER_AREA`; `INTER_LINEAR` or `INTER_CUBIC` would be better for enlarging.
3. **Only width / only height:** change one dimension and keep the other → aspect ratio not preserved, so the image stretches.
4. **Specific size (200×200):** forces a square → also stretched.
5. **With fx / fy:** `python cv.resize(img, None, fx=0.6, fy=0.6, interpolation=cv.INTER_LINEAR)` `None` means "no exact size; use the scale factors." This avoids the width/height mix-up entirely.

How resizing actually works (not covered in later sessions)

Resizing doesn't directly merge or delete pixels. OpenCV **creates a new empty grid** at the new size, then fills each new pixel by asking: **"Where does this pixel land in the original image, and what color is there?"** The mapped position usually falls **between** original pixels (say, column 1.6). **Interpolation** is the rule for choosing a value there.

Example with one row of pixels: `[10, 20, 30, 40]`

1. `INTER_NEAREST`: copy the closest pixel

- Position 1.6 → nearest pixel is 2 → value **30**.
- **Upscaling to 8 pixels** repeats each pixel: `[10,10,20,20,30,30,40,40]` → big blocky squares.
- **Downscaling to 2 pixels** keeps some pixels and **drops** the rest: `[10, 30]`.
- Fastest, but looks the roughest.

2. `INTER_LINEAR`: blend the 2 nearest pixels by distance

- Position 1.6 sits between 20 (at 1) and 30 (at 2), 60% of the way toward 30.
- Value = 20 × 0.4 + 30 × 0.6 = **26**
- In 2D it blends the **4 nearest pixels** (a 2×2 square), so it's also called **bilinear**.
- Upscaling looks smooth instead of blocky. This is the default.

3. `INTER_CUBIC`: blend 16 nearby pixels (4×4) with a smooth curve

- More neighbors → sharper, smoother edges. Best for **upscaling**, but slower.

4. `INTER_AREA`: average every pixel the new pixel covers

- Shrinking `[10,20,30,40]` to 2 pixels: each new pixel covers 2 old ones → `[15, 35]`.
- Nothing is dropped; every original pixel contributes to the average.
- Best for **downscaling**. Nearest neighbor would skip pixels, causing jagged edges and flickering patterns.

Shrinking	Enlarging
`INTER_AREA`: **merges** pixels by averaging	`INTER_LINEAR` / `INTER_CUBIC`: **creates** in-between pixels by blending neighbors
`INTER_NEAREST`: **drops** pixels	`INTER_NEAREST`: **repeats** pixels, looks blocky

Distortions to note (original is 332 wide × 300 tall)

Resize	New size (w × h)	Distortion
Only width	100 × 300	Squeezed **horizontally**: the dog looks thin and tall.
Only height	332 × 200	Squashed **vertically**: the dog looks short and wide.
Both	200 × 200	**Smaller** and slightly squeezed sideways, since the original wasn't square (332 → 200 shrinks the width more than 300 → 200 shrinks the height).

Takeaway: when width and height are scaled by different amounts, shapes stretch. To resize without distortion, scale both by the same factor, for example `fx=fy=0.5`.

Common mistakes when writing the resize exercise

Mistake	What happens	Fix
`int(100, h)` to make a size	`TypeError: int() can't convert non-string with explicit base`. `int()` takes one value; a second argument is read as a number base.	Use a tuple: `(100, h)`
Using `h`, `w` without defining them	`NameError`	`h, w, _ = img.shape` first
`cv.imgshow`	`AttributeError`	`cv.imshow`
`cv.imshow('title', both_resized)` where `both_resized = (200, 200)`	No error, but the window shows **only its title**: OpenCV treats `(200, 200)` as a tiny 2-pixel image.	Pass the resized **image**, not the size

A working version:

```
h, w, _ = img.shape

resized_w = (100, h)       # (width, height): only the width changes
resized_h = (w, 200)       # only the height changes
both_resized = (200, 200)

resized_img_w = cv.resize(img, resized_w, interpolation=cv.INTER_AREA)
resized_img_h = cv.resize(img, resized_h, interpolation=cv.INTER_AREA)
resized_img_both = cv.resize(img, both_resized, interpolation=cv.INTER_AREA)

cv.imshow('w resized', resized_img_w)
cv.waitKey(7000)
cv.imshow('h resized', resized_img_h)
cv.waitKey(6000)
cv.imshow('both dimensions resized', resized_img_both)
cv.waitKey(3000)
cv.destroyAllWindows()
```

If a window still looks empty, click on it (on Windows an OpenCV window sometimes doesn't paint until it gets focus) or use `cv.waitKey(0)`.

Exercise in the notebook: resize with different interpolation methods and compare. Upscale a lot (for example 4×) with `INTER_NEAREST`, then with `INTER_CUBIC`, and the difference becomes obvious.

E. Cropping

```
Cropped_img = img[80:200, 100:200]
```

Cropping is **just NumPy slicing**. There's no special OpenCV function for it.

- `80:200` → rows (y) 80 to 199
- `100:200` → columns (x) 100 to 199
- Result: a 120 × 100 piece of the image.

Remember the order is `img[y, x]`: **rows first, then columns**.

⚠️ Slicing returns a **view**, not a copy. If you modify `Cropped_img`, the original `img` changes too. Use `img[80:200, 100:200].copy()` for an independent crop.

Exercise: split the image into blocks (patches). Loop over the image in steps and slice out tiles:

```
patch_h, patch_w = 100, 100
H, W, _ = img.shape
for y in range(0, H, patch_h):
    for x in range(0, W, patch_w):
        patch = img[y:y+patch_h, x:x+patch_w]
        # show or save the patch
```

F. Rotating

Two steps.

Step 1: build the rotation matrix

```
height, width, _ = img.shape
center = (width/2, height/2)    # (x, y): rotate around the middle
rotate_matrix = cv.getRotationMatrix2D(center=center, angle=45, scale=1)
```

- **angle > 0** → **counter-clockwise**, angle < 0 → clockwise.
- `scale=1` keeps the size. Something like 0.5 would also shrink the image.
- The result is a **2×3 matrix**: `[[ 0.707, 0.707, -57.4], [-0.707, 0.707, 161.3]]` The left 2×2 part is the rotation (cos 45° = sin 45° ≈ 0.707). The third column is a **shift** that keeps the image rotating around its center instead of the top-left corner.

Step 2: apply it

```
rotated_image = cv.warpAffine(src=img, M=rotate_matrix, dsize=(width, height))
```

This moves every pixel according to the matrix. Note that `dsize=(width, height)` is correctly width-first here.

What you'll notice:

- **The corners get cut off** and **black triangles appear**. The output canvas is the same size as the original, so the rotated corners stick out, and empty areas are filled with 0 (black). The pyimagesearch "rotate image" link at the end of the notebook shows how to enlarge the canvas.
- At 90° with a non-square image (332×300), parts get cut off too, because width and height would need to swap.
- 180° turns the image upside down.

Part 3: Python OOP refresher

This notebook has no classes, but you **use** objects everywhere.

Class vs. object: a class is a blueprint and an object is something built from it.

```
class Dog:                          # blueprint
    def __init__(self, name, age):  # constructor, runs when you create a Dog
        self.name = name            # attribute (data stored in the object)
        self.age = age

    def bark(self):                 # method (a function that belongs to the class)
        return f"{self.name} says woof!"

d = Dog("Rex", 3)   # create an object (an "instance")
d.name              # 'Rex'   ← attribute (no parentheses)
d.bark()            # 'Rex says woof!' ← method call (with parentheses)
```

- `self` means "this particular object." Python passes it automatically: you write `d.bark()`, not `d.bark(d)`.
- `__init__` is the constructor.
- **Inheritance:** `class Puppy(Dog):` gets everything `Dog` has and can add to it or override it.
- **Dunder names** (double underscore, like `__init__` or `__version__`) are special names Python itself uses.

Where this shows up in the notebook:

Code	OOP concept
`img`	An **object**, an instance of the class `numpy.ndarray`
`type(img)`	Asks "which class is this object from?"
`img.shape`	An **attribute** (no parentheses)
`img.copy()`	A **method** (with parentheses)
`cv.__version__`	An attribute of the `cv2` **module** (modules are objects too)
`cv.resize(...)`	A plain **function** inside a module, not a method

You'll need real OOP later (for example in PyTorch: `class MyModel(nn.Module):`).

Cheat-sheet

```
img = cv.imread(path)                    # load (returns None if the path is wrong!)
img.shape                                 # (H, W, C) → height FIRST
img[y, x]                                 # one pixel → [B, G, R]
cv.imshow("t", img); cv.waitKey(0); cv.destroyAllWindows()
cv.resize(img, (W, H))                    # OpenCV size → width FIRST
cv.resize(img, None, fx=0.5, fy=0.5)      # scale by factors (no distortion)
img[y1:y2, x1:x2]                         # crop → rows (y) first
M = cv.getRotationMatrix2D((W/2, H/2), angle, 1)  # +angle = counter-clockwise
cv.warpAffine(img, M, (W, H))
```

Remember:

- Images are NumPy arrays, and OpenCV uses BGR.
- `.shape` is (h, w), but OpenCV sizes are (w, h).
- Cropping is just slicing.
- `./images/` is the `images` folder inside the current directory, and the trailing `/` matters.
- `waitKey(0)` waits forever; press a key in the image window to continue.
- Shrink with `INTER_AREA`, enlarge with `INTER_LINEAR` / `INTER_CUBIC`.